

Threat Modeling Assistant: LLM-Augmented STRIDE and LINDDUN Threat Modeling with OWASP MASVS v2 Controls for Android Applications

Etienne Didier Dansou^a, Ali Darkaoui^a, Salah Eddine Bouaanane^a, Yao Landry Dossah^a

^aUnder the supervision of Pr. Mohamed LACHGAR

National School of Applied Sciences of Marrakech (ENSA Marrakech), Cadi Ayyad University, Marrakech, Morocco. Field: Cybersecurity and Embedded Telecommunication Systems Engineering.

Abstract

Threat Modeling Assistant (TMA) is an open-source web platform that automates the generation of structured STRIDE and LINDDUN threat models for Android mobile applications. Given an optional APK binary and a textual application description, the tool extracts security-relevant artifacts—permissions, exported components, and API endpoints—via Androguard static analysis, then invokes a configurable Large Language Model (LLM) at temperature 0.2 to elicit six targeted missing-context questions before synthesizing a JSON threat model mapped to 20 OWASP MASVS v2 controls. Six LLM providers are supported, including fully offline operation via Ollama and a mock provider requiring no API key. The platform is deployable with a single `docker compose up -build` command. Source code is available at https://github.com/Eti-Hack/Threat_Modeling_Assistant.git.

Keywords: threat modeling, STRIDE, LINDDUN, OWASP MASVS, Android security, static analysis, large language models, DevSecOps

Required Metadata

Current code version

Nr.	Code metadata description	Information
C1	Current code version	v1.0.0
C2	Permanent GitHub link to code/repository	https://github.com/Eti-Hack/Threat_Modeling_Assistant.git
C3	Legal Code License	MIT License
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python 3.12, FastAPI 0.111, Androguard 4.1, Pydantic v2, React 18, Vite 5, TailwindCSS 3.4, Framer Motion 11, Docker
C6	Compilation requirements, operating environments & dependencies	Python 3.12+, Node.js 18+, Docker (recommended); pip: fastapi, uvicorn, androguard, pydantic-settings, python-multipart; npm: react, vite, tailwindcss, framer-motion, lucide-react; Optional: Ollama (offline LLM) or Anthropic/OpenAI API key
C7	Link to developer documentation/manual	https://github.com/Eti-Hack/Threat_Modeling_Assistant/blob/main/README.md
C8	Support email for questions	aicybersec1337@gmail.com

Table 1: Code metadata (mandatory).

1. Motivation and Significance

Threat modeling is a structured process for identifying security and privacy threats during the design phase of software development, before vulnerabilities are embedded in code. Standards bodies including OWASP, NIST SP 800-154, and ISO/IEC 27034 recognize it as a foundational secure-development activity. Despite this recognition, adoption in mobile development teams remains limited. Shostack [1] identifies three recurring barriers: the time required to produce a model manually (2–5 days for a non-trivial application), the specialist expertise needed to apply frameworks such as STRIDE [3] or LINDDUN [4], and the absence of tooling integrated with mobile development workflows.

For Android applications, the challenge is compounded by the specificity of the mobile attack surface. Android’s component model—activities, services, broadcast receivers, and content providers—introduces threat categories that generic tools do not address. The OWASP Mobile Application Security Verification Standard v2 (MASVS v2) [2], with its seven control families (STORAGE, CRYPTO, AUTH, NETWORK, PLATFORM, CODE, RESILIENCE), provides the most authoritative catalog of Android-specific mitigations, but mapping a threat to the correct MASVS controls still requires domain expertise.

STRIDE [3], introduced at Microsoft in 1999, enumerates six threat categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. **LINDDUN** [4], developed at KU Leuven, extends threat modeling to privacy: Linkability, Identifiability, Non-repudiation, Detectability, Disclosure of information, Unawareness, and Non-compliance— dimensions particularly relevant under GDPR.

Recent work on LLM-assisted security analysis [5, 6] demonstrates that modern large language models can generate accurate threat catalogs when provided with sufficient application context. For Android apps, part of that context is encoded in the APK binary (permissions, exported components, API endpoint strings in the DEX string table); the rest must be elicited from the development team.

Threat Modeling Assistant (TMA) bridges this gap by combining Androguard-based APK static analysis with LLM-driven interactive context elicitation and structured STRIDE or LINDDUN generation, with every threat entry mapped to specific OWASP MASVS v2 controls.

Related Work

Microsoft Threat Modeling Tool [7] is a desktop application for STRIDE analysis on manually drawn Data Flow Diagrams. It provides no mobile-specific threat library and no LLM assistance. **OWASP Threat Dragon** [8] is an open-source web tool supporting STRIDE on manually constructed diagrams, with no APK analysis or MASVS mapping. **IriusRisk** [9] is a commercial enterprise platform with broader standards coverage, but it does not ingest APK binaries and its enterprise pricing restricts academic use. **ThreatSpec** [10] derives threat models from source-code annotations; it does not operate on compiled APKs and provides no MASVS mapping.

None of these tools simultaneously (1) extract security context from an APK binary without source code, (2) support both STRIDE and LINDDUN, (3) map every threat to specific OWASP MASVS v2 controls, and (4) use an LLM for both context elicitation and structured generation.

2. Software Description

2.1. Software Architecture

TMA follows a three-tier client-server architecture, illustrated in Figure 1.

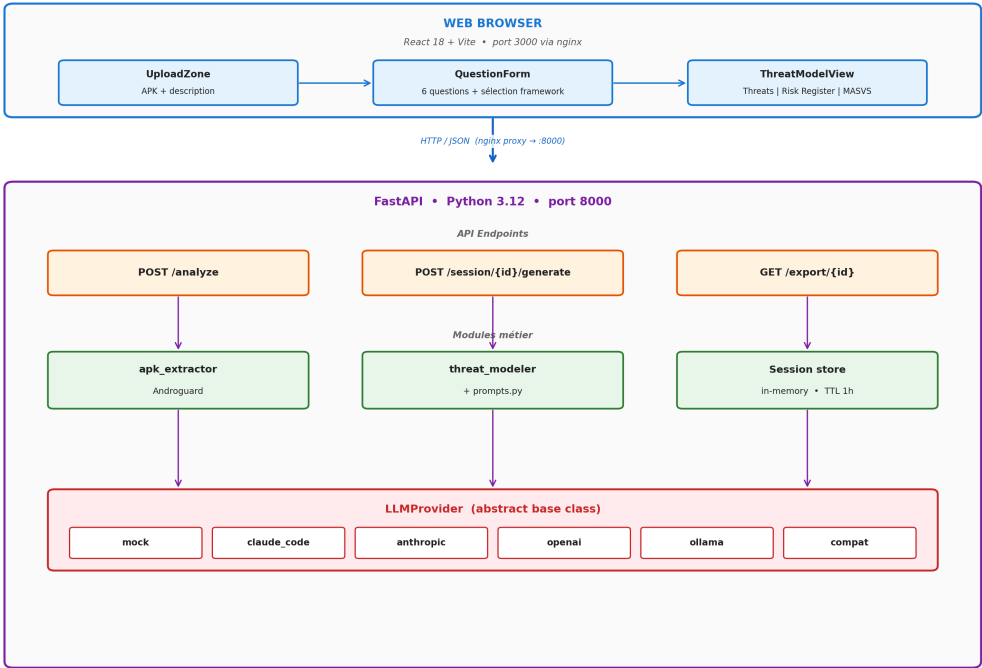


Figure 1: Component-level architecture of Threat Modeling Assistant. The nginx reverse proxy routes `/api/` requests to the FastAPI backend while serving the React SPA at the root path.

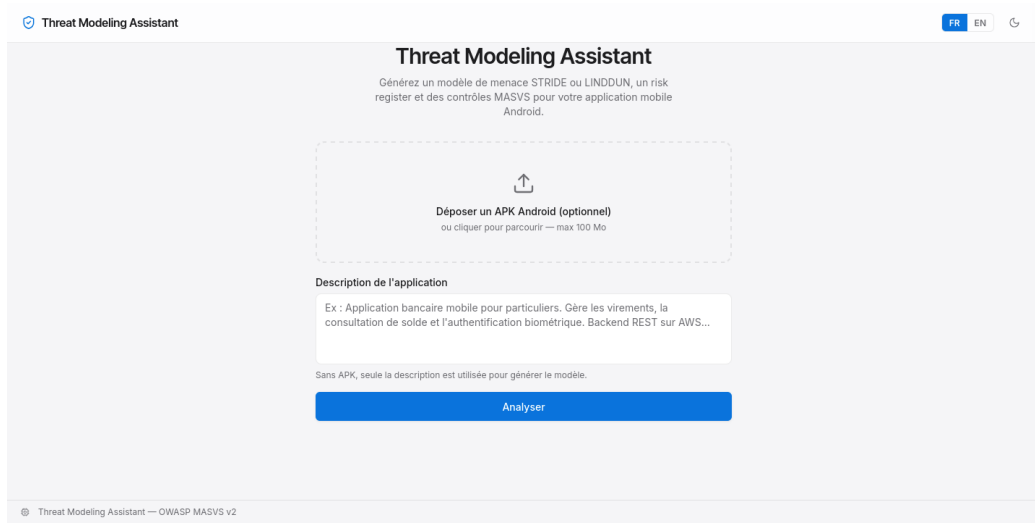


Figure 2: Landing screen of Threat Modeling Assistant. Users can optionally upload an Android APK (up to 100 MB) and provide a natural-language description of the application before launching the analysis.

Backend (Python 3.12 / FastAPI [11]) exposes six REST endpoints. The `apk_extractor` service invokes Androguard [12] inside a temporary-directory context manager to parse the APK manifest and scan DEX string tables for URL patterns. Session state is kept in an in-memory dictionary; an asyncio background coroutine started in the FastAPI `lifespan` context discards sessions after one hour. The APK binary is written to `/tmp/threatmodel/<uuid>.apk` and deleted immediately after extraction. All prompts are defined in the single module `llm/prompts.py`; inline prompt strings elsewhere in the codebase are prohibited by architecture policy.

LLM Abstraction Layer is built on the abstract base class `LLMProvider` with a single async `complete()` method. The concrete provider is resolved at startup by a factory function using lazy local imports, so unused SDK dependencies are never loaded. Table 2 lists the six supported providers.

LLM_PROVIDER	Implementation	Required configuration
mock (default)	Static JSON response	None — no API key required
claude_code	Subprocess via stdin (JSONL)	Claude Code CLI + <code>claude login</code>
anthropic	Anthropic SDK (async)	<code>ANTHROPIC_API_KEY</code>
openai	OpenAI SDK (async)	<code>OPENAI_API_KEY</code>
ollama	HTTP to local Ollama	<code>OLLAMA_BASE_URL</code>
openai_compatible	OpenAI SDK + custom URL	<code>LLM_BASE_URL</code> , <code>LLM_MODEL</code>

Table 2: Supported LLM providers and required configuration. Switching providers requires only one environment variable change; no code modification is needed.

The `claude_code` provider invokes the Claude Code CLI with flags `-print -verbose -output-format stream-json`, passing the prompt via stdin to avoid command-line length limits. The JSONL output is parsed to extract the `{"type": "result", "subtype": "success"}` event; a fallback concatenates content `[] .text` fields from `type:assistant` events if the result event is absent.

Frontend (React 18, Vite [13], TailwindCSS) is a three-step single-page application guiding the user from APK upload through question answering to results visualization. All visual values are CSS custom properties in `tokens.css`, enabling full dark-mode support via `data-theme="dark"` on `<html>` without any external theming library. FR/EN language selection is implemented as a React context with `localStorage` persistence; the `language` field is propagated to all API calls so the LLM generates output in the user’s selected language.

Deployment uses Docker Compose: the backend image builds from `python:3.12-slim` with `libxml2-dev` for Androguard; the frontend image uses a two-stage build (`node:20-alpine` to bundle, `nginx:alpine` to serve). The full stack starts with:

```

1 cp .env.example .env           # default: LLM_PROVIDER=
   mock
2 docker compose up --build
3 # Application available at http://localhost:3000

```

Listing 1: Full-stack deployment command.

2.2. Software Functionalities

1. APK Static Analysis and Context Extraction (POST /api/analyze).

The endpoint accepts `multipart/form-data` with three fields: `file` (APK binary, optional, max 100 MB), `description` (free-text string), and `language`

(`fr` or `en`). When an APK is provided, `apk_extractor.py` calls Androguard’s `AnalyzeAPK()` and assembles an `AppContext` Pydantic model containing: package name, version, all declared permissions, exported activities/services/receivers/providers (components with `android:exported=true` or an `<intent-filter>` without an explicit `exported=false`), and API endpoint strings extracted by the pattern `https?://[\s"’<>]{6,}` applied to all DEX string tables and XML resource files. If no APK is provided, `AppContext` is populated from the description only; the endpoint is identical in either case.

`generate_missing_questions(ctx, lang)` then sends a structured prompt to the active LLM provider (`max_tokens=800`, `temperature=0.2`) requesting a JSON array of six targeted questions. A bilingual fallback list is returned if JSON parsing fails. The endpoint response includes `session_id`, `app_context`, and `missing_questions`.

2. Threat Model Generation (POST `/api/session/{id}/generate`).

The request body carries `framework` (STRIDE or LINDDUN), `answers` (dict mapping each question to the user’s response), and `language`. The LLM is called with `max_tokens=4096`, `temperature=0.2`, and `response_format={"type": "json_object"}`. The prompt includes: a formatted `AppContext` summary, the 20 OWASP MASVS v2 controls with their exact IDs, scoring rules (`risk_score` 1–9; CRITICAL 8–9, HIGH 6–7, MEDIUM 4–5, LOW 1–3), and a strict JSON output schema. When `lang="en"`, the suffix “*Write ALL generated text fields ... in ENGLISH*” is appended to the prompt.

The response is parsed into a `ThreatModel` Pydantic model containing `List[ThreatEntry]` and `List[RiskEntry]`. JSON parsing is resilient: a regex first isolates the outermost `{...}` block to handle models that wrap JSON in markdown fences. Malformed individual entries are silently skipped to maximize partial recovery.

3. Results Visualization.

The `ThreatModelView` React component renders three tabs: *Threats* (cards sorted by likelihood CRITICAL→LOW, with category badge, asset, actor, attack, impact, and MASVS control pills); *Risk Register* (sortable table with color-coded score column: red ≥ 8 , orange ≥ 6 , yellow ≥ 4 , green < 4); and *Mitigations* (entries grouped by primary MASVS v2 category: MASVS-STORAGE, MASVS-CRYPTO, MASVS-AUTH, MASVS-NETWORK, MASVS-PLATFORM, MASVS-CODE, MASVS-RESILIENCE).

4. Export and Session Management.

`GET /api/export/{id}` returns the full `ThreatModel` as a JSON download (`Content-Disposition: attachment`). Sessions are cleaned after one hour by an asyncio coroutine. `DELETE /api/session/{id}` supports early manual cleanup.

2.3. Sample Code Snippets

Listing 2 shows the LLMProvider abstract interface. All six concrete implementations inherit from this class. The factory function uses lazy local imports so that, for example, the anthropic SDK is never imported when LLM_PROVIDER=ollama.

```
1 from abc import ABC, abstractmethod
2 from dataclasses import dataclass
3
4 @dataclass
5 class LLMMessage:
6     role: str      # "user" | "assistant" | "system"
7     content: str
8
9 @dataclass
10 class LLMResponse:
11     content: str
12     model: str = ""
13
14 class LLMProvider(ABC):
15     @abstractmethod
16     async def complete(
17         self,
18         messages: list[LLMMessage],
19         max_tokens: int = 2048,
20         temperature: float = 0.2,
21         response_format: dict | None = None,
22     ) -> LLMResponse: ...
```

Listing 2: LLMProvider abstract interface (llm/base.py).

Listing 3 shows the language override mechanism. The structural prompt remains in French (the design language of the prompt corpus) while an appended instruction directs the model to generate all user-visible text fields in English when requested.

```
1 _LANG_OVERRIDE_EN = (
2     "\nIMPORTANT: Write ALL generated text fields (
3         asset names, "
4         "actor descriptions, attack descriptions, impact,
5         risk "
6         "descriptions, mitigations) in ENGLISH."
7 )
8
9 def _lang_suffix(lang: str) -> str:
```


8

```
return _LANG_OVERRIDE_EN if lang == "en" else ""
```

Listing 3: Language override suffix (llm/prompts.py).

3. Illustrative Examples

3.1. End-to-End Workflow: Banking Application

We demonstrate TMA on a fictional banking application described as: “*Mobile banking app for retail customers handling wire transfers, balance inquiries, and biometric authentication. REST backend on AWS. Uses Firebase Analytics.*” No APK is uploaded; the description alone is used to illustrate the description-only path.

Step 1 — Context analysis. The POST /api/analyze endpoint returns a session. The LLM generates the following six questions (language=en, temperature 0.2):

1. *What type of sensitive data is stored locally (account numbers, transaction history, biometric templates)?*
2. *Is biometric authentication implemented via Android BiometricPrompt API or a custom mechanism?*
3. *Does the app implement certificate pinning for the AWS REST backend?*
4. *How are authentication tokens stored and what is their expiration policy?*
5. *Which Firebase services beyond Analytics are integrated (Crashlytics, Remote Config, Cloud Messaging)?*
6. *Is the app targeted at retail consumers, business customers, or both?*

Threat Modeling Assistant
FR EN

< Back
1 Context
2 Questions
3 Threat Model

Additional information

The AI has identified missing information to refine the threat model. Answer the questions below.

- Quelles catégories de données sensibles l'application manipule-t-elle (PII, données financières, médicales, géolocalisation, identifiants d'entreprise) ?
Your answer (optional)...
- Quel mécanisme d'authentification est utilisé (mot de passe local, OAuth2/OIDC, biométrie, certificat client, SSO d'entreprise) et comment les tokens de session sont-ils stockés et invalidés ?
Your answer (optional)...
- Quelle est l'architecture backend (API REST/GraphQL, cloud provider, base de données) et l'application fonctionne-t-elle hors ligne avec synchronisation différée ?
Your answer (optional)...
- Quel est le profil des utilisateurs cibles (grand public, employés d'entreprise, mineurs, professionnels de santé) et l'application est-elle distribuée via un MDM ou uniquement le Play Store ?
Your answer (optional)...
- Les données sensibles stockées localement sont-elles chiffrées (Android Keystore, EncryptedSharedPreferences, SQLCipher) et quels fichiers ou bases de données sont écrits sur le stockage externe ?
Your answer (optional)...
- Quelles intégrations tierces sont présentes dans l'application (SDK analytics, Firebase, passerelle de paiement, publicité, SSO, bibliothèques open source) et quel niveau de confiance leur est accordé ?
Your answer (optional)...

Modeling framework

STRIDE
Spoofing · Tampering · Repudiation · Information Disclosure · DoS · EoP

LINDDUN
Privacy-focused modeling — Linkability · Identifiability · Disclosure...

Generate threat model

Figure 3: Context analysis step. After the POST `/api/analyze` endpoint returns a session, the LLM (`language=en`, `temperature=0.2`) generates six targeted questions covering sensitive data, authentication, backend architecture, user profile, local storage and third-party integrations. The user selects the modeling framework (STRIDE or LINDDUN) before launching threat model generation.

Step 2 — STRIDE threat model. The user answers the questions, selects STRIDE, and submits. Listings 4 and 5 show one representative `ThreatEntry` and its corresponding `RiskEntry` from the generated model.

```

1 {
2   "id": "S3",
3   "category": "Information Disclosure",
4   "asset": "Authentication tokens stored in
5     SharedPreferences",
6   "actor": "Co-installed malicious app on a rooted
7     device",
8   "attack": "Unencrypted SharedPreferences are
9     readable directly from
10    the data partition; replaying stolen

```

```

8         tokens grants full
          account access without user credentials
          .",
9     "impact": "Confidentiality breach: complete account
          takeover.",
10    "likelihood": "HIGH",
11    "risk_score": 7,
12    "masvs_controls": ["MASVS-STORAGE-1", "MASVS-STORAGE
          -2", "MASVS-AUTH-2"]
13 }

```

Listing 4: Representative STRIDE ThreatEntry (Information Disclosure).

```

1 {
2     "threat_id": "S3",
3     "risk": "Session hijacking via token exfiltration
          from unencrypted storage.",
4     "score": 7,
5     "priority": "HIGH",
6     "mitigation": "Store tokens in Android Keystore-
          backed
7                     EncryptedSharedPreferences (Jetpack
                      Security).
8                     Enforce short expiration (15-30 min)
                      with sliding refresh.
9                     Implement root detection (MASVS-
                      RESILIENCE-1).",
10    "masvs_ref": "MASVS-STORAGE-1, MASVS-STORAGE-2,
          MASVS-AUTH-2,
11                MASVS-RESILIENCE-1"
12 }

```

Listing 5: Corresponding RiskEntry for threat S3.

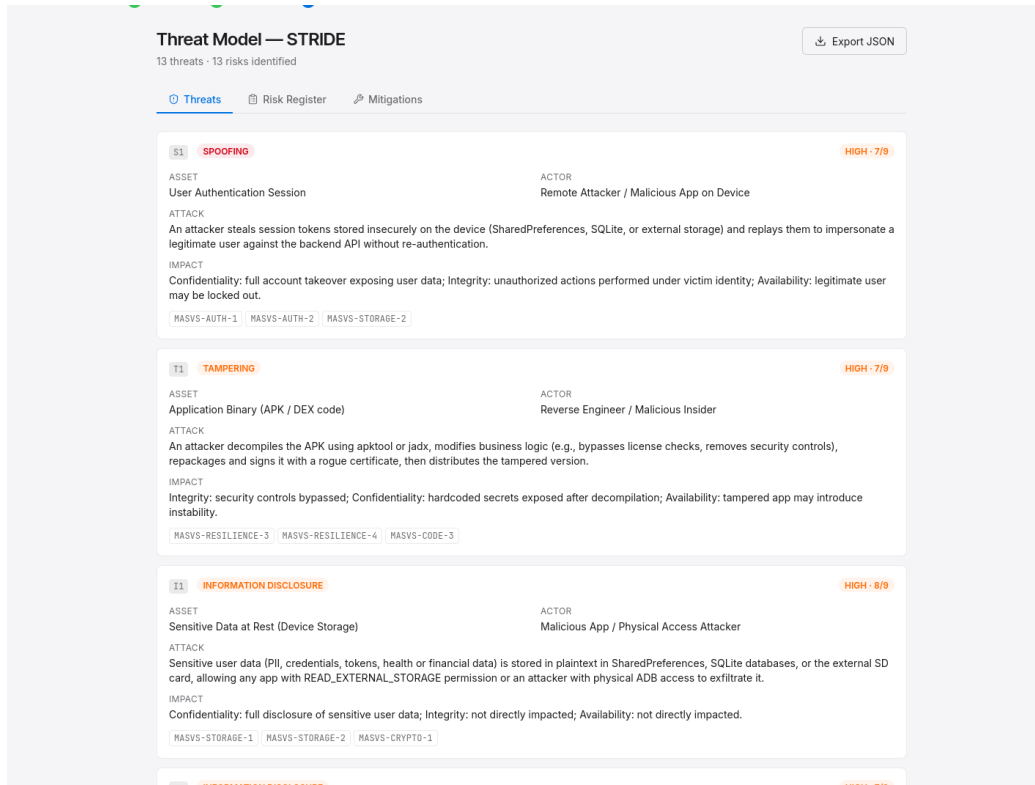


Figure 4: Step 2 — STRIDE threat model. After the user answers the questions, selects STRIDE and submits, the application displays the generated threat list. Each entry exposes the asset, actor, attack scenario, impact assessment and the corresponding MASVS controls. Listings ?? and ?? show one representative `ThreatEntry` and its corresponding `RiskEntry` from the generated model.

Step 3 — Mitigations tab. Entry S3 appears under the *MASVS-STORAGE* group alongside other storage-related threats, providing a consolidated remediation view per control family so that a developer can address all storage findings in a single engineering pass.

3.2. Reproducing the Example

```

1 git clone https://github.com/Eti-Hack/
  Threat_Modeling_Assistant.git && cd
  Threat_Modeling_Assistant
2 cp .env.example .env # LLM_PROVIDER=mock by
  default
3 docker compose up --build
4 # Open http://localhost:3000, type the description,
  submit.
```

```

5
6 # To use a real LLM:
7 echo "LLM_PROVIDER=anthropic"           >> .env
8 echo "ANTHROPIC_API_KEY=sk-ant-..."   >> .env
9 docker compose restart backend

```

Listing 6: Commands to reproduce the illustrative example.

With `LLM_PROVIDER=mock` the full interface—including all three result tabs and JSON export—is exercised without any API key or network access.

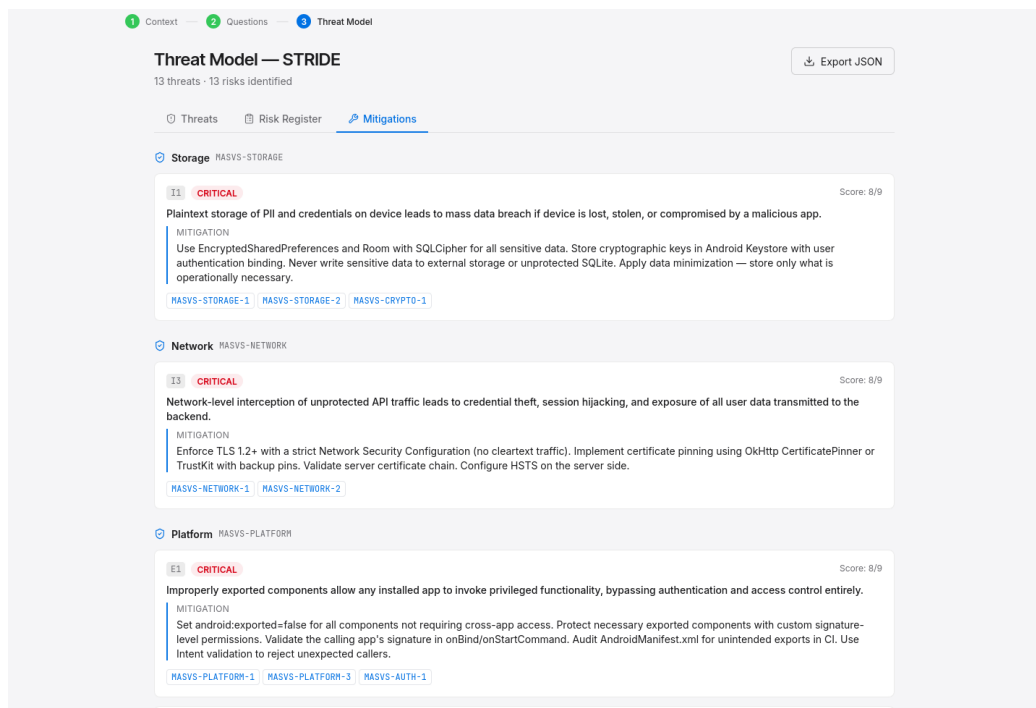


Figure 5: Step 3 — Mitigations tab. Threat entries are regrouped by MASVS control family (MASVS-STORAGE, MASVS-NETWORK, MASVS-PLATFORM, ...) and each group lists the consolidated remediation guidance. This view allows a developer to address all findings related to a given control family in a single engineering pass.

4. Impact

4.1. New Research Questions

TMA makes it practical to study, at scale, the quality gap between LLM-generated and expert-authored STRIDE/LINDDUN models for Android applications. Because the prompts, parsing logic, and provider abstraction are modular, researchers can substitute different LLMs, vary the prompt

strategy, or inject different APK-derived context features and measure their effect on threat coverage and MASVS control precision. The structured `ThreatModel` JSON output enables downstream analyses: correlation between permission sets and threat category distributions, or longitudinal tracking of threat surface evolution across app releases. The bilingual (FR/EN) output additionally supports cross-linguistic consistency studies of LLM-generated security content.

4.2. Comparison with Existing Tools

Table 3 positions TMA against the four most widely used threat modeling tools for the features most relevant to Android mobile application security.

Feature	TMA	MS TMT [7]	Threat Dragon [8]	IriusRisk [9]	Threat
APK binary analysis	✓	✗	✗	✗	
STRIDE support	✓	✓	✓	✓	
LINDDUN support	✓	✗	✗	Part.	
OWASP MASVS v2 mapping	✓	✗	✗	✓	
LLM-assisted generation	✓	✗	✗	✗	
Interactive context elicit.	✓	✗	✗	✗	
Risk register output	✓	✓	✓	✓	
JSON export	✓	✗	✓	✓	
Docker deployment	✓	✗	✓	Cloud	
Fully offline	✓	✓	✓	✗	
Open-source	✓	✗	✓	✗	
FR/EN bilingual UI+output	✓	✗	✗	✗	

Table 3: Feature comparison between TMA and representative threat modeling tools. TMA is the only open-source tool combining APK binary analysis, STRIDE/LINDDUN support, OWASP MASVS v2 control mapping, and LLM-assisted generation in a single deployable platform. *Part.* = partial support.

4.3. Impact on Development Workflows

TMA integrates into DevSecOps pipelines as a pre-sprint or pre-release activity. A developer or security champion can upload the application APK (or provide a textual description for a new project), answer the LLM-generated questions in approximately 10 minutes, and obtain a prioritized risk register with MASVS v2 control references. The JSON export can be committed to the project repository and `diffed` between releases to track threat surface changes over time.

The `mock` provider enables CI integration: a pipeline can call `/api/analyze` and `/session/{id}/generate` to verify endpoint availability and response

schema validity without incurring LLM API costs. The provider abstraction allows organizations with strict data-residency requirements to deploy entirely on-premises with Ollama and a locally-hosted model such as `qwen2.5-coder:7b`, with no data leaving the organization’s network.

5. Conclusions

Threat Modeling Assistant addresses a gap in the Android mobile security toolchain: the absence of an open-source, LLM-assisted tool combining binary-level APK analysis, dual-framework STRIDE/LINDDUN support, and automated mapping to OWASP MASVS v2 controls. By externalizing context elicitation to a structured LLM dialogue, TMA reduces the expert time required to initiate a threat model while preserving the analyst’s domain knowledge as the authoritative input.

The provider-agnostic LLM abstraction, the zero-configuration mock fallback, and the single-command Docker deployment minimize the operational barrier to adoption. The FR/EN bilingual interface and output extend the tool’s reach to Francophone development teams.

Future work will address three directions: (1) automatic Data Flow Diagram (DFD) generation from the `AppContext` to provide a visual complement; (2) integration into CI/CD pipelines as a pre-merge threat model diff gate; and (3) fine-tuning smaller open-source LLMs (7B–13B parameters) on curated STRIDE/LINDDUN corpora to improve structured JSON output quality in fully-offline Ollama deployments.

Acknowledgements

The authors thank Pr. Mohamed Lachgar (Cadi Ayyad University) for supervision and guidance throughout this project, and Pr. Omar Achbaro, head of the Cybersecurity and Embedded Telecommunication Systems Engineering program at ENSA Marrakech, for academic support.

References

- [1] A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014.
- [2] OWASP Foundation, “Mobile Application Security Verification Standard v2.0.0,” <https://mas.owasp.org/MASVS/>, 2023.
- [3] L. Kohnfelder and P. Garg, “The threats to our products,” *Microsoft Interface*, Apr. 1999.

- [4] M. Deng, K. Wuyts, R. Scandariato, B. Preneel, and W. Joosen, “A privacy threat analysis framework: supporting the elicitation and fulfillment of privacy requirements,” *Requirements Engineering*, vol. 16, no. 1, pp. 3–32, 2011.
- [5] I. Happe and J. Cito, “Getting pwned by AI: Penetration testing with large language models,” in *Proc. 31st ACM ESEC/FSE*, 2023.
- [6] M. A. Ferrag et al., “Revolutionizing Cyber Threat Detection with Large Language Models,” *IEEE Access*, vol. 12, 2024.
- [7] Microsoft, “Microsoft Threat Modeling Tool,” <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool>, 2024.
- [8] OWASP Foundation, “Threat Dragon,” <https://owasp.org/www-project-threat-dragon/>, 2024.
- [9] IriusRisk, “Automated Threat Modeling Platform,” <https://www.iriusrisk.com/>, 2024.
- [10] J. Bristow, “ThreatSpec: Continuous threat modeling,” <https://threatspec.org/>, 2023.
- [11] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com/>, 2024.
- [12] A. Desnos and G. Gueguen, “Androguard: Reverse Engineering, Malware and Goodware Analysis of Android Applications,” <https://github.com/androguard/androguard>, 2024.
- [13] Vite contributors, “Vite: Next Generation Frontend Tooling,” <https://vitejs.dev/>, 2024.

Current executable software version

Nr.	Software metadata description	Information
S1	Current software version	v1.0.0
S2	Permanent link to executables of this version	https://github.com/Eti-Hack/Threat_Modeling_Assistant/releases/tag/v1.0.0
S3	Legal Software License	MIT License
S4	Computing platforms / Operating Systems	Linux, macOS, Windows (via Docker); Python 3.12, Node.js 18
S5	Installation requirements & dependencies	Docker and Docker Compose (recommended). Alternatively: Python 3.12+ with pip dependencies listed in <code>backend/requirements.txt</code> and Node.js 18+ with npm dependencies in <code>frontend/package.json</code> . Optional: Ollama for offline LLM inference, or an Anthropic/OpenAI API key for cloud inference.
S6	Link to user manual	https://github.com/Eti-Hack/Threat_Modeling_Assistant/blob/main/README.md
S7	Support email for questions	aicybersec1337@gmail.com

Table 4: Executable software metadata (mandatory).